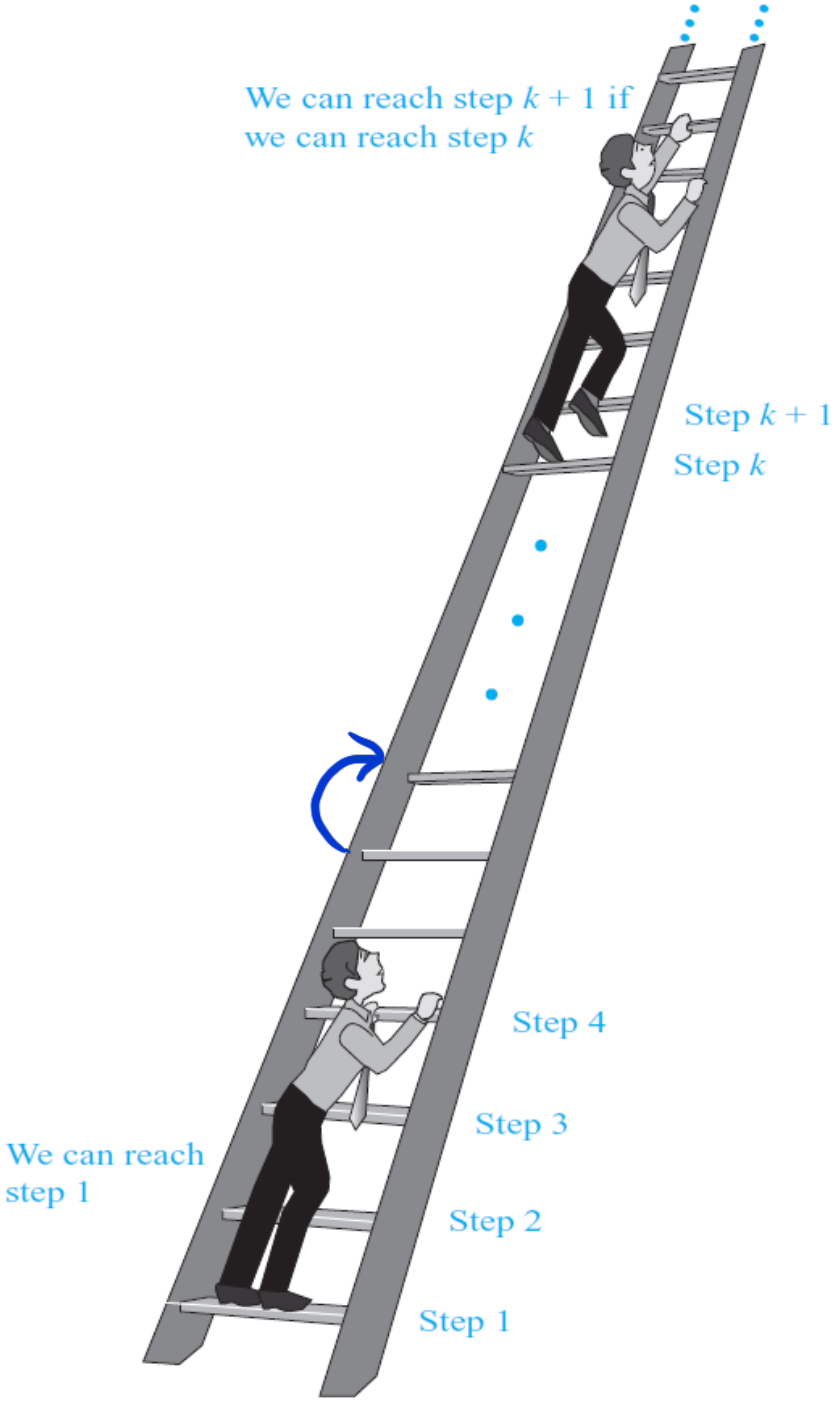
Mathematical Induction

(Paragraph 5.1)



1. We can reach the first rung of the ladder.
2. If we can reach a particular rung of the ladder, then we can reach the next rung.

Question: Can we reach every rung?

Conclusion: Yes

PRINCIPLE OF MATHEMATICAL INDUCTION:

To prove that $\forall n P(n)$ is true, we complete two steps:

BASIS STEP: $P(1)$ is true.

INDUCTIVE STEP:

$\forall k(P(k) \rightarrow P(k+1))$ is true.

Proving conjectures

Find the sum of the first n positive odd integers

Solution: $n=1 \Rightarrow S=1$, $n=2 \Rightarrow S=1+3=4$

$n=3 \Rightarrow S=1+3+5=9$, $n=4 \Rightarrow S=1+3+5+7=16$ ...

hypothesis $\boxed{S(n)=n^2}$

proof: $P(n)=$ "$S(n)=n^2$"

B.S.: $P(1)=$ "$S(1)=1$" $=$ true

I.S.: $\forall k\,(P(k)\to P(k+1)) \equiv$ true

Assume that $P(k)=1$, that is $S(k)=k^2$

$S(k+1)=\underbrace{1+3+\ldots+2k-1}_{k^2}+2(k+1)-1=$

$=k^2+2k+2-1=k^2+2k+1=(k+1)^2 \Rightarrow P(k+1)=1$

$1+3+5+\ldots+(2k-1)$

$\Rightarrow \forall k\,(P(k)\to P(k+1))=1 \Rightarrow$ By P.M.I $\forall n\,P(n)=1$ ∎

Proving inequalities

Use mathematical induction to prove the inequality

$$n < 2^n$$

for all positive integers n.

Solution: Let $P(n) =$ "$n < 2^n$".
BASIS STEP: $P(1) =$ "$1 < 2^1$" $= 1$
INDUCTIVE STEP:
Assume that $P(k) =$ "$k < 2^k$" $= 1$
$P(k+1)$: $k+1 < 2^k + 1 < 2^k + 2^k = 2 \cdot 2^k \leq 2^{k+1}$
$\Rightarrow$ $P(k+1)$ is true $\Rightarrow$ $\forall k\,(P(k) \rightarrow P(k+1)) = 1$
$\Rightarrow$ By P. M. I $\Rightarrow$ $\forall n\, P(n)$ is true
that is $\forall n$ $n < 2^n$

The Number of Subsets of a Finite Set

Use mathematical induction to show that if S is a finite set with n elements, where n is a nonnegative integer, then S has 2^n subsets.

Solution: $P(n)$ = "a set with n elements has 2^n subsets"

BASIS STEP: $P(0)$ = "the empty set has 1 subset" = 1 since $\emptyset \subset \emptyset$

INDUCTIVE STEP:

Assume that $P(k)$ is true

Let T be a set with $k+1$ elements

$T = S \cup \{a\}$ $(|S| = k) \Rightarrow S$ has 2^k subsets

For each subset $X \subset S$ we can construct exactly two subsets from T: X and $X \cup \{a\}$

$\Rightarrow$ the number of subsets of T is $2^k \cdot 2 = 2^{k+1}$

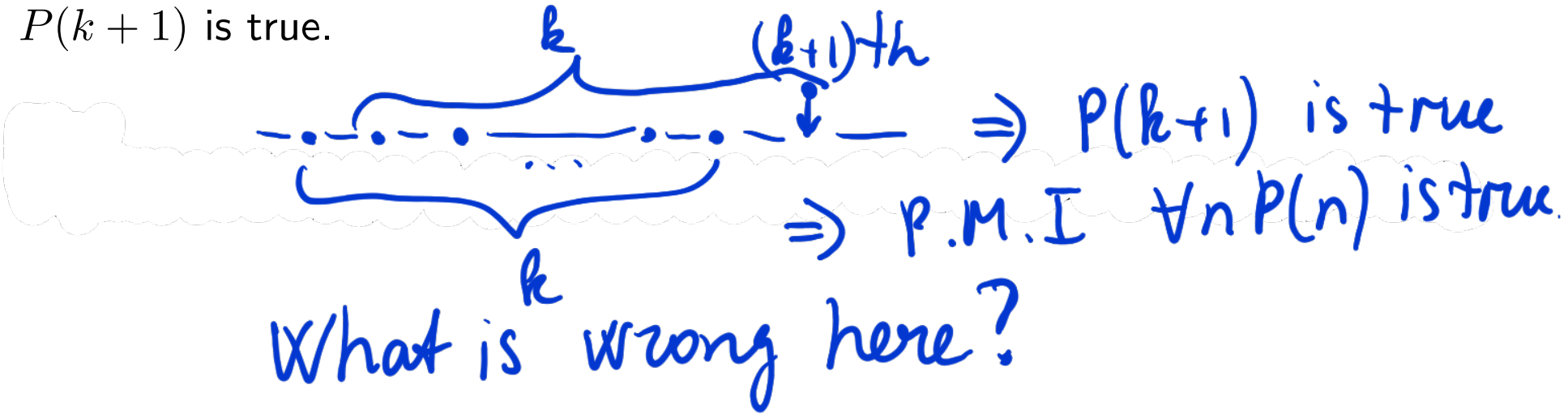
$\Rightarrow P(k+1)$ is true $\Rightarrow \forall k\,(P(k) \to P(k+1)) = 1 \Rightarrow$

by P.M.I $\Rightarrow \forall n\, P(n)$ is true. ☐

Be careful with the induction

Given a set of n points. Then these points lay on one line.

Proof: Let $P(n) =$ "n points lay on one line".
BASIS STEP: Clearly, one point lays on one line so $P(1)$ is true.
INDUCTIVE STEP: Assume that $P(k)$ is true.
Consider a set of $k+1$ points. Consider a subset of k points. Then these lay on a line. Consider another subset of k points. Then these lay on a line. The intersection of these sets contain $k-1$ points, so these lines are the same. Hence, $P(k+1)$ is true.



Homework:

- Solve all odd exercises at p. 329 until #23 and also numbers 31,33,39,41,43,49,51.

Answer Q1

Strong Induction and Well-Ordering

(Paragraph 5.2)

STRONG INDUCTION: To prove that $P(n)$ is true for all positive integers n, we complete two steps:
BASIS STEP: We verify that $P(1)$ is true.
INDUCTIVE STEP: We show that

$$\forall k(P(1) \wedge P(2) \wedge \ldots \wedge P(k) \rightarrow P(k+1))$$

is true.

Proof of factorization theorem

Theorem

Show that if n is an integer greater than 1, then n is prime or can be written as the product of primes.

Solution: Let $P(n) =$ "n is prime or product of primes"

BASIS STEP: $P(2) =$ "2 is prime" $= 1$

INDUCTIVE STEP:

Assume that $P(j) = 1$ for $2 \le j \le k$

if $k+1$ is prime, then $P(k+1) = 1$

If $k+1$ is composite and so $k+1 = a \cdot b$, $2 \le a, b \le k+1$

$\Rightarrow$ a and b are primes or products of primes

Thus, $k+1 = a \cdot b$ is product of primes $\Rightarrow$

$\Rightarrow P(k+1) = 1 \Rightarrow$ By S.I. $\forall n\, P(n) = 1$ ∎

The well-ordering property

The validity of both the principle of mathematical induction and strong induction follows from a fundamental axiom of the set of integers, the well-ordering property:

Every nonempty set of non-negative integers has a least element.

The well-ordering property, the principle of mathematical induction, and strong induction are all equivalent (and equivalent to the axiom of choice).

Theorem

Prove that if a is an integer and d is a positive integer, then there are integers q and r with $0 \leq r < d$ and $a = dq + r$.

Solution: $S = \{a - dq \geq 0 : q \in \mathbb{Z}\}$

By the well ord. pr. $\Rightarrow$ there is a list element in this set S achieved for q_0, $r = a - dq_0 \geq 0$

assume $r \geq d$, then $r - d = a - dq_0 - d = a - d(q_0 + 1) \in S$

also $a - d(q_0 + 1) \leq a - dq_0$, but $a - dq_0$ was least

$\Rightarrow$ contradict. $\Rightarrow$ $r < d$ ∎

Homework:

- Solve exercises # 1,3,25,29,37,41 at p. 341-344.

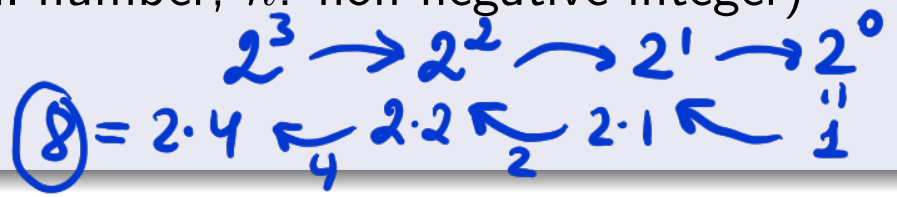
Recursive Algorithms (The Merge Sort)

(Paragraph 5.4)

An algorithm is called recursive if it solves a problem by reducing it to an instance of the same problem with smaller input.

Example 1 (power)

procedure power(a: non-zero real number, n: non-negative integer)
if $n = 0$ **then return** 1
else return $a \cdot$ power$(a, n-1)$



Example 2 (gcd)

procedure gcd(a, b: non-negative integers with $a < b$)
if $a = 0$ **then return** b
else return gcd$(b \bmod a, a)$

gcd(8,12) → gcd(4,8) → gcd(0,4) → return 4

Example 3 (linear search)

procedure search(i, n, x: integers, $1 \le i \le n$)
if $a_i = x$ **then return** i
else if $i = n$ **then return** 0
else return search$(i+1, n, x)$

$a_1, a_2, \ldots, a_n$
$i=1$ if $a_1 \neq x$, then $i=2$ if $a_2 \neq x$
$\Rightarrow i=3$ ...

Proving Recursive Algorithms Correct

Prove that the algorithm in Example 1 is correct.

Solution: $P(n)$ = "The alg. returns correctly a^n"

B.S.: If $n=0$, the alg returns $1 \Rightarrow P(0)=1$

I.S.: Assume that the alg. returns a^k corr.

$\text{power}(a, k+1) = a \cdot \underbrace{\text{power}(a, k)}_{\text{correct}} = a \cdot a^k = a^{k+1}$

$\Rightarrow$ the alg. returns a^{k+1} cor. $\Rightarrow P(k+1)=1$

$\Rightarrow$ By P.M.I. $\forall n P(n)$ is true ∎

Time complexity of recursion (power)

Algorithm 1

procedure power(a: non-zero real number, n: non-negative integer)
if $n = 0$ **then return** 1
else return $a \cdot$ power$(a, n-1)$ $= T(n-1)$

Let $T(n)$ the compl. of the alg for input n.

$$T(n) = T(n-1) + C, \quad n > 0, \quad T(0) = 2.$$

$$T(n) = T(n-1) + C = (T(n-2) + C) + C = T(n-2) + 2C =$$
$$= \ldots = T(1) + (n-1)C = T(0) + nC = \Theta(n)$$

Algorithm 2

procedure power(a: non-zero real number, n: non-negative integer)
if $n = 0$ **then return** 1
else if n is even
  then $y :=$power$(a, n/2)$
    return $y \cdot y$
  else return $a \cdot$ power$(a, n-1)$

$2^3 = 2 \cdot 2^2$ $\quad$ $2^4 = 2^2 \cdot 2^2$ $\quad$ // y // y

$T(n)$ time complexity

$$T(n) = \begin{cases} T\left(\frac{n}{2}\right) + c_1 & , n \text{ is even} \\ T(n-1) + c_2 & , n \text{ is odd} \end{cases}, \quad T(0) = 1$$

let n is odd

$T(n) = T(n-1) + c_2 = T\left(\frac{n-1}{2}\right) + c_1 + c_2$, for simpl. $n = 2^k$

$T(n) = T\left(\frac{n}{2}\right) + c_1 = T\left(\frac{n}{4}\right) + 2c_1 = T\left(\frac{n}{8}\right) + 3c_1 = \ldots$

$\ldots = T\left(\frac{n}{2^k}\right) + kc_1 = T(1) + c_1 \log n = c + c_1 \log n =$

$= \Theta(\log n)$

Time complexity of recursion (search)

Linear search

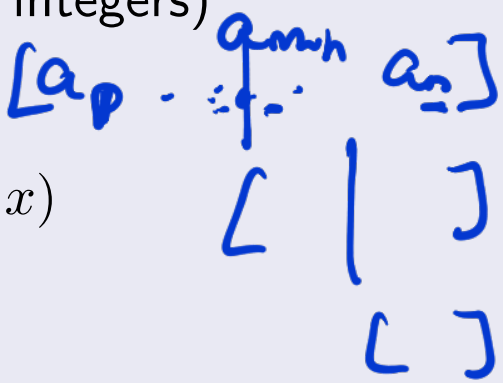
procedure search(a: array of integers, n, x: integers, $1 \leq i \leq n$)
if $a_n = x$ **then return** n
else if $n = 0$ **then return** 0
else return search$(a, n-1, x)$

$T(n) = T(n-1) + C$, $n > 0$, $T(0) = 1$

$\Rightarrow T(n) = O(n)$

Binary search

procedure search(a: sorted array of integers, ℓ, r, x: integers)
if $r \geq \ell$ **then** $mid := \lfloor (r + \ell)/2 \rfloor$
  if $a_{mid} = x$ **then return** mid
  else if $a_{mid} > x$ **then return** search$(a, \ell, mid - 1, x)$
    else return search$(a, mid + 1, r, x)$
else return 0
launch search$(a, 1, n, x)$



$T(n)$, for convenience $2^k = n$

$$T(n) = T\left(\frac{n}{2}\right) + c_1 = T\left(\frac{n}{4}\right) + 2c_1 = \dots$$

$$\dots = T\left(\frac{n}{2^k}\right) + kc_1 = T(1) + \log n \cdot c_1 =$$

$$= C + c_1 \log n = \Theta(\log n)$$

Time complexity of recursion (sorting)

Bubble sort

procedure bubbleSort(a: array of integers, n: integer)
if $n = 1$ **then return** a
else for $i := 1$ **to** $n - 1$
    if $a_i > a_{i+1}$ **then** swap(a_i, a_{i+1})
  bubbleSort$(a, n - 1)$

$[a_1, a_2, \dots, a_n]$

if $a_1 > a_2$ then $[a_2, a_1, a_3, \dots, a_n]$

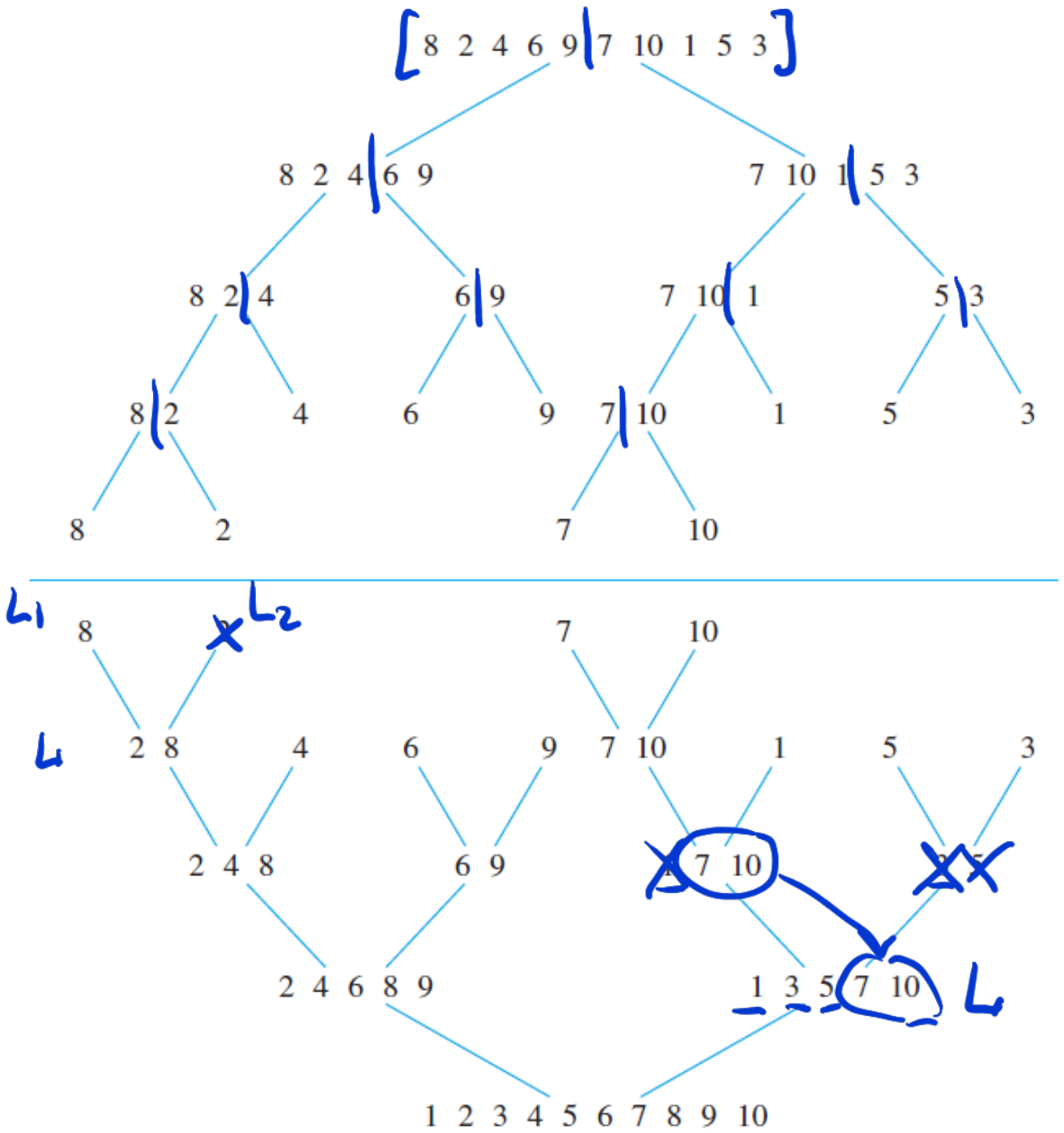
after this loop $[a_1, a_2, \dots, a_{n-1}, (a_n)]$ — a_n largest

$T(n) = T(n-1) + C_1 n + C_2, n > 0, T(1) = 1$

$T(n) = T(n-1) + C_1 n + C_2 = T(n-2) + C_1(n-1) + C_1 n + 2C_2 =$

$= \dots = T(1) + C_1 \cdot n^2 + C_2' n + C_3 = \Theta(n^2)$

Merge sort


8 2 4 6 9 7 10 1 5 3
8 2 4 6 9
7 10 1 5 3
8 2 4
6 9
7 10 1
5 3
8 2
4
6
9
7 10
1
5
3
8
2
7
10
L1
L2
L
8
2 8
4
6
9
7 10
1
5
3
2 4 8
6 9
7 10
2 4 6 8 9
1 3 5 7 10
L
1 2 3 4 5 6 7 8 9 10

Answer Q2

A Recursive Merge Sort

procedure mergesort($L = a_1, \ldots, a_n$)
if $n > 1$ **then** $m := \lfloor n/2 \rfloor$
    $L_1 := a_1, a_2, \ldots, a_m$
    $L_2 := a_{m+1}, a_{m+2}, \ldots, a_n$
    $L :=$merge(mergesort(L_1), mergesort(L_2))
return L

splitting

Merging Two Lists

procedure merge(L_1,L_2 : sorted lists)
$L :=$ empty list
while L_1 and L_2 are both nonempty
    remove smaller of first elements of L_1 and L_2 from its list
    put it at the right end of L
    if this removal makes one list empty
        then remove all elements from the other list and append them to L
return L

merging

Complexity of the Merge Sort algorithm

$n = 2^k$

$$T(n) = 2T\left(\frac{n}{2}\right) + cn, \quad n > 0, \quad T(1) = 1$$

$$T(n) = 2T\left(\frac{n}{2}\right) + cn = 2\left(2T\left(\frac{n}{4}\right) + c\frac{n}{2}\right) + cn$$

$$= 4T\left(\frac{n}{4}\right) + 2cn = \ldots =$$

$$= 2^k T\left(\frac{n}{2^k}\right) + kcn = nT(1) +$$

$$+ cn\log n = O(n\log n)$$

↖ the best possible sorting alg.

Master Theorem (Paragraph 8.3)

Theorem

Let T be an increasing function that satisfies the recurrence relation

$$T(n) = aT(n/b) + cn^d$$

$a=2, b=2$
$d=1$

whenever $n = b^k$, where k is a positive integer, $a \geq 1$, b is an integer greater than 1, and c and d are real numbers with c positive and d non-negative. Then

$$T(n) = \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log n) & \text{if } a = b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

$2 = 2^1$

Proof is by induction left as an exercise.

Fast Matrix Multiplication

$$T(n) = 7T(n/2) + 15n^2/4$$

$a=7, b=2, d=2$

addition of $n \times n$ matrix

$7 > 2^2$

is $O(n^{\log_2 7 < 2.81})$

< 3

Homework:

- Solve *odd* exercises starting at p. 370.

Answer Q3